QALight

Frontend-розробка

Заняття 9. Модуль Flex. Таблиці.

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Вісь, напрямки, розподіл простору, вирівнювання, зовнішні відступи, розмір елемента, коефіцієнти розтягування та стиснення елементів.
Рядки та комірки таблиці, заголовок, угруповання рядків та колонок таблиці. Стовпчик таблиці, вирівнювання таблиці, межі таблиці.
Реалізація Flex-таблиці.

Flex

вісь, напрямки, розподіл простору, вирівнювання, зовнішні відступи, розмір елемента, коефіцієнти розтягування та стиснення елементів.

Модель flexbox

Теорія: flex

Модель flexbox (**Flexible Box - гнучкий бокс**) є одновимірним і простим способом розподілу простору, розміщення та вирівнювання елементів у контейнері.

За допомогою flexbox **контейнер** має можливість змінювати ширину та висоту своїх елементів, їх порядок та визначати кількість місця, яку може займати конкретний елемент у контейнері.

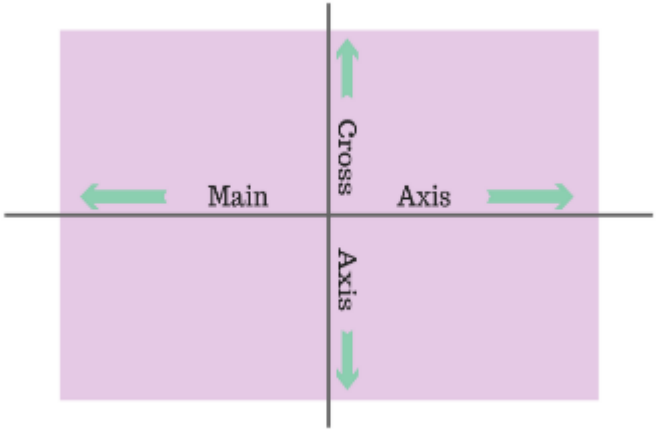
Розташування елементів html не має значення. Елементи автоматично шикуються у кілька рядків/стовпчиків, займаючи все доступне місце.

Flexbox адаптовано для написання справа наліво **rtl** (**right-to-left**) та зліва направо **ltr** (**left-to-right**). Тобто у браузерах з локалью rtl всі елементи будуть автоматично розміщені у реверсному порядку.

Головна вісь і поперечна вісь

Теорія: flex

У flexbox є дві осі - **головна** вісь (**main axis**) і поперечна вісь (**cross axis**), що перпендикулярна головній.



- **flex-direction** встановлює головну вісь.
- **justify-content** визначає розташування елементів вздовж головної.
- **align-items** визначає розташування елементів вздовж поперечної.
- **align-self** вказує, як окремий flex-елемент розиашований на поперечній осі.

Головна вісь

Теорія: flex

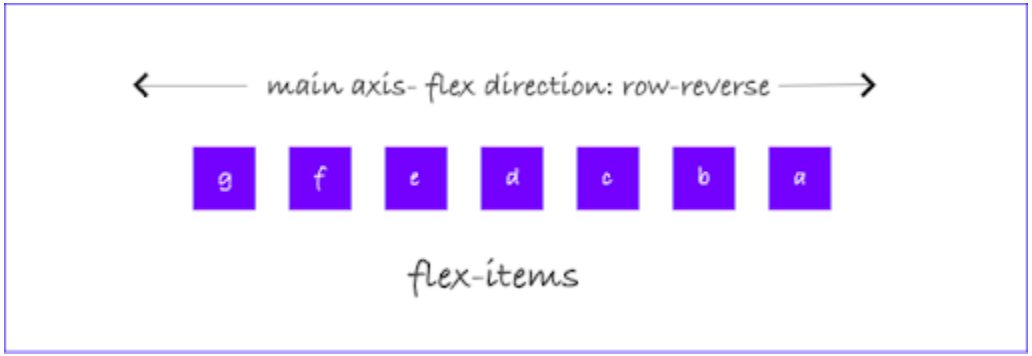
Головна вісь визначається напрямком, у якому розміщуються flex-елементи. Початок і кінець цієї осі називають **головним-початком** і **головним-кінцем**. У браузерах, де англійська мова є мовою за замовчуванням, елементи розміщуються зліва направо в контейнері.



Значення row-reverse

Теорія: flex

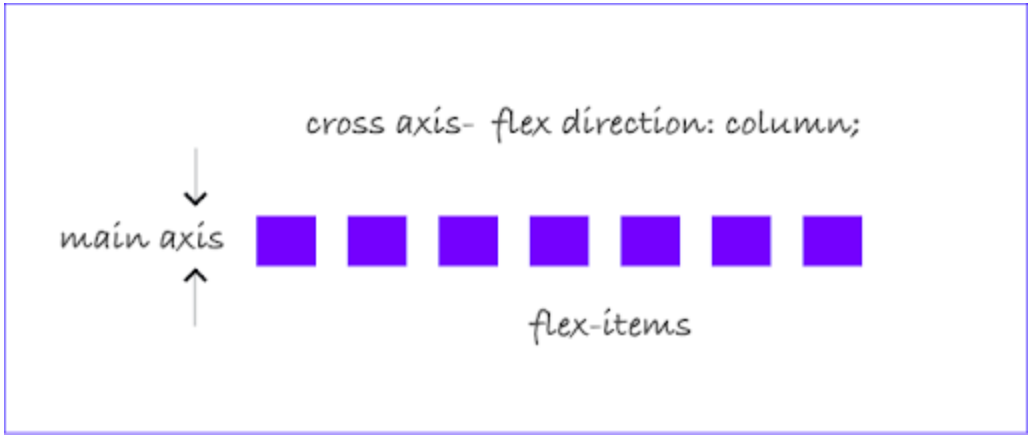
Значення r**ow-reverse** викладає елементи в контейнері flex у зворотному порядку.



Поперечна вісь

Теорія: flex

Поперечна вісь проходить перпендикулярно головній осі.



flex-контейнер

Теорія: flex

Щоб почати використовувати flexbox, потрібно спочатку визначити flex-контейнер. Це батьківський елемент, який утримує дітей у вашому коді. Це може бути div, span, nav тощо.

Батьківський контейнер стає гнучким, якщо для властивості display встановлено значення flex.

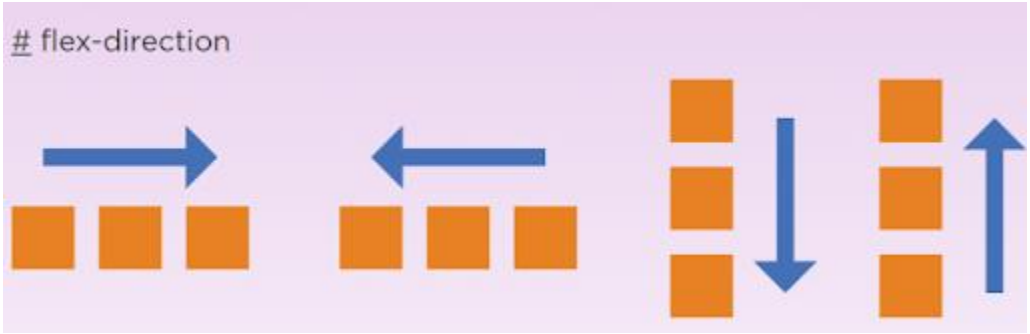
```
.parent {
  display: flex;
}
```

Ви також можете налаштувати батьківський контейнер на display:**inline-flex**; Це створить гнучкий контейнер, який є вбудованим, а не блоком.

Властивість flex-direction

Теорія: flex

Властивість **flex-direction** визначає напрямок, у якому розташовується головна вісь.



Властивість flex-direction

Теорія: flex

flex-direction: row|row-reverse|column|column-reverse **-** властивість батьківського flex-контейнеру.

- **row (за замовчуванням)**: зліва направо для ltr, справа наліво для rtl;
- row-reverse: справа наліво(ltr), зліва направо(rtl);
- column: аналогічно row, зверху вниз;
- column-reverse: аналогічно row-reverse, знизу вверх.

```
.navbar .navbar-nav {
  flex-direction: row;
}
.navbar-nav {
  flex-direction: column;
}
```

Властивість flex-wrap

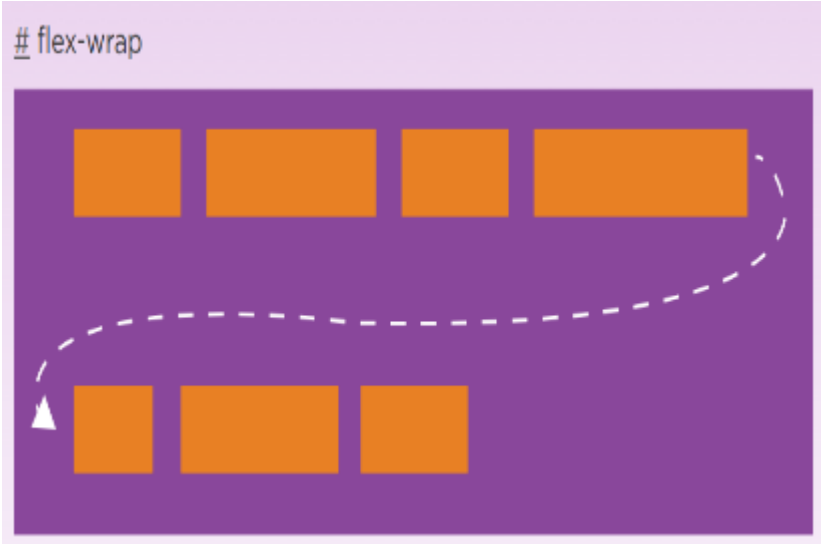
Теорія: flex

Якщо є набір flex-елементів, які занадто широкі для їхнього контейнера, вони переповнять його. Можна розмістити їх на кількох рядках у контейнері за допомогою **flex-wrap**. Властивість flex-wrap дозволяє елементам у контейнері займати більше одного рядка.

flex-wrap: nowrap | wrap | wrap-reverse

Значенням за замовчуванням flex-wrap є **nowrap**, елементи будуть займати **лише один рядок**.

```
.row {
   display: flex;
   flex-wrap: wrap;
}
```



Властивість flex-flow

Практика: flex

Також можна комбінувати flex-wrap з flex-direction для елементів, використовуючи скорочену властивість **flex-flow**.

flex-flow: flex-direction || flex-wrap

Значенням за замовчуванням є **row nowrap**.

```
.navbar {
  display: flex;
  flex-flow: row nowrap;
}
```

Властивість flex

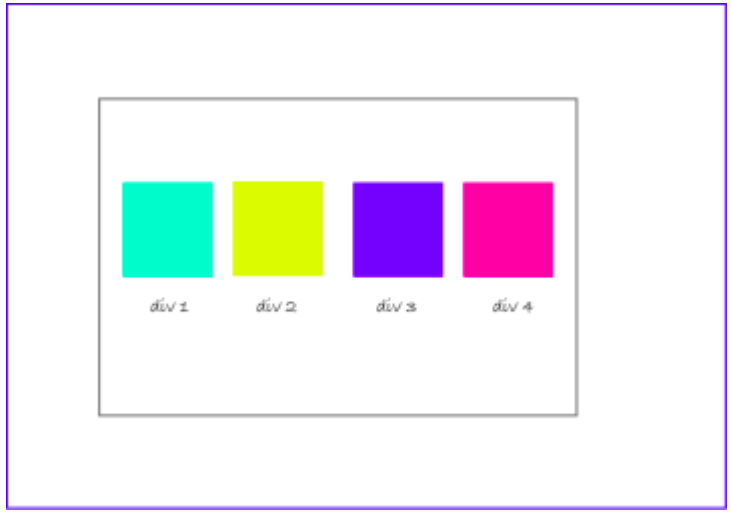
Практика: Медіа-функції

Властивість **flex** | **inline-flex** - це скорочена властивість, що дозволяє одночасно встановити 3 окремі властивості:

```
.child {
        [flex-grow flex-shrink flex-basis]
}
```

Стилі за замовчуванням:

```
.child {
  flex: 0 1 auto;
}
```



Властивість FLEX-GROW

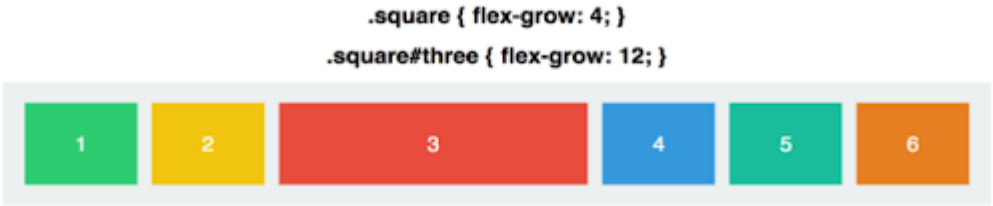
Практика: fontr

flex-grow встановлює коефіцієнт зростання, що дозволяє гнучкому елементу збільшуватися та заповнювати доступний простір. Значення **flex-grow приймає лише ціле число**.

flex-grow має значення 0 за замовчуванням. Кожен елемент залежить від розміру вмісту в ньому.

Якщо потрібно, щоб лише один з елементів зростав більше, ніж інші, наприклад втричі, необхідно зробити наступне:

```
.child-three {
    flex: 3 1 auto;;
 }
```

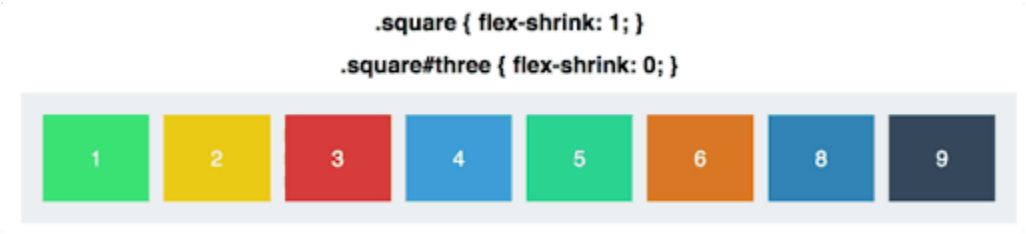


Властивість flex-shrink

Практика: flex

Друге значення в скороченні flex - це **flex-shrink.** Значенням за замовчуванням є 1, що означає: **Займай однакову кількість місця в будь-який час**:

```
.child {
  flex: 0 1 auto; /* flex-shrink = 1 */
}
```



Властивість flex-basis

Практика: flex

flex-basis - це останнє значення у скороченні flex, контролює **розмір елемента** за замовчуванням. Flex-basis впливає на розмір елемента на головній осі. Flex-basis по черзі визначає ширину або висоту, залежно від flex-direction.

```
.parent {
  display: flex;
}
.child {
  flex: 0 1 auto;
}
```

flex-basis контролює, наскільки великим буде елемент вздовж головної осі, перш ніж відбудеться будь-яке зростання або стискання. Flex-grow визначає, наскільки він зросте пропорційно до однотипних елементів, а flex-shrink визначає, наскільки він зменшиться.\

Властивість justify-content

Практика: flex

Justify-content розподіляє елементи flex вздовж головної осі в контейнері.

Існують різні значення, які ми можемо використовувати з властивістю justify-content.

justify-content: flex-start | flex-end | center | space-between | space-around;

flex-start: значення елементів за замовчуванням. Елементи викладаються з початку flex-контейнера.
flex-end: елементи викладаються з кінця flex-контейнера.

center: предмети розташовуються в центрі контейнера.

space-between: елементи мають рівний простір між собою, але не між ними і контейнером. Іншими словами, перший і останній елемент не мають місця між ними та контейнером.

space-around: елементи мають рівну кількість простору між ними, перший і останній елемент мають деякий простір між ними. Цей проміжок становить ПОЛОВИНУ суми між двома елементами.

Властивість align-items

Практика: flex

Властивість **align-items: flex-start | flex-end | center | baseline | stretch дозволяє** вирівняти, розташувати та розподілити елементи у гнучкому контейнері вздовж поперечної осі. Так само, як justify-content, властивість має різні значення.

- stretch: значення за замовчуванням. Елементи займають весь простір на поперечній осі.
- flex-start: елементи викладаються від початку згинання на поперечній осі.
- flex-end: елементи викладаються на кінці flex-контейнера.
- **center**: предмети викладаються в центрі контейнера вздовж поперечної осі.
- **baseline**: елементи вирівнюються таким чином, щоб базові лінії були рівними.

Властивість CSS align-items вирівнює flex-елементи вздовж поперечної (перпендикулярної) осі flex-контейнера. Якщо головна вісь горизонтальна (flex-direction: row), то align-items вирівнює елементи по вертикалі. Детальніше про це написано на MDN Web Docs.Основні значення властивості stretch (значення за замовчуванням): елементи розтягуються на всю висоту контейнера.flex-start: вирівнює елементи по початку поперечної осі (вгорі).flex-end: вирівнює елементи по кінцю поперечної осі (внизу). center: вирівнює елементи по центру поперечної осі. baseline: вирівнює елементи по їх базовій лінії тексту.

Властивість align-content

Практика: flex

CSS-властивість align-content визначає, як рядки флекс-елементів вирівнюються вздовж поперечної осі всередині flex-контейнера, коли є вільний простір.

Головні особливості

- **Поперечна вісь:** Працює перпендикулярно до головної осі (наприклад, по вертикалі, якщо ряди йдуть горизонтально).
- **Багаторядковість:** Потребує значення flex-wrap: wrap або wrap-reverse, щоб елементи переносилися на нові рядки та з'явилося кілька осей.
- **Вільне місце:** Працює лише тоді, коли висота контейнера більша за сумарну висоту всіх вкладених рядків.

Популярні значення

- flex-start — притискає рядки до початку поперечної осі.
- flex-end — притискає рядки до кінця осі.
- center — центрує рядки всередині контейнера.
- space-between — розподіляє рядки так, що перший стає на початку, останній у кінці, а простір між ними рівно ділиться.
- space-around — ділить вільний простір порівну навколо кожного рядка.
- stretch — розтягує рядки на весь вільний простір (значення за замовчуванням).

Створення адаптивної панелі навігації

Практика: navbar

Flexbox – один із найчистіших способів вирівнювання елементів по горизонталі на екранах робочих столів та їх вертикального розташування на мобільних пристроях. Ця структура розділяє панель навігації на логічне розташування: логотип бренду ліворуч та список посилань навігації праворуч.\

```
<nav class="navbar">
      <div class="nav-logo">
         <a href="#">BrandLogo</a>
      </div>
      <ul class="nav-links">
         <li><a href="#">Home</a></li>
         <li><a href="#">About</a></li>
         <li><a href="#">Services</a></li>
         <li><a href="#">Contact</a></li>
      </ul>
  </nav>
```

Базові стилі панелі навігації

Практика: navbar

```css
/* Navbar Base Styles  Базові стилі панелі навігації*/
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #1a1a1a;
  padding: 1rem 2rem;
}

.nav-logo a {
  color: #ffffff;
  font-size: 1.5rem;
  font-weight: bold;
  text-decoration: none;
}
```

Контейнер посилань

Практика: navbar

```css
/* Links Container as a Flex Box */
.nav-links {
  display: flex;
  list-style: none;
  gap: 1.5rem; /* Evenly spaces individual links */
}

.nav-links a {
  color: #ffffff;
  text-decoration: none;
  font-size: 1rem;
  transition: color 0.3s ease;
}

.nav-links a:hover {
  color: #00adb5; /* Accent color on hover */
}
```

Table

Рядки та комірки таблиці, заголовок, угруповання рядків та колонок таблиці. Стовпчик таблиці, вирівнювання таблиці, межі таблиці. Реалізація Flex-таблиці.

Таблиці

Практика: Таблиці

Таблиці HTML дозволяють розміщувати такі дані, як текст, зображення, посилання, інші таблиці тощо, у рядки та стовпці клітинок.

Таблиці HTML створюються за допомогою тегу <table>, у якому тег <tr> використовується для створення рядків таблиці, а тег <td> використовується для створення клітинок даних. Елементи під <td> звичайні та вирівняні за лівим краєм за умовчанням

```
<table border = "1">
 <tr>
    <td>Row 1, Column 1</td>
    <td>Row 1, Column 2</td>
 </tr>
  <tr>
    <td>Row 2, Column 1</td>
    <td>Row 2, Column 2</td>
 </tr>
</table>
```

Елемент thead

Практика: Таблиці

Елемент **<thead>** визначає контекст для заголовка. Елемент **<tbody>** визначає область, де буде розміщено табличний вміст. Вони визначають область, але самі по собі не показують вміст. Елемент **<tfoot>** існує для надання підсумкової інформації.

```
<thead></thead>
<tbody></tbody>
<tfoot></tfoot>
```

Якщо таблиця містить блок **<thead>**, блок <tbody> має бути після нього.

```
<thead>  <tr>  <td colspan = "4">This is the head of the table</td>  </tr>  </thead>
```

Елемент HTML **<tbody>** інкапсулює набір рядків таблиці (елементи <tr>), вказуючи, що вони складають тіло таблиці (<table>).

Якщо елементи <tr> вказано поза елементом <tbody>, ці елементи будуть інкапсульовані елементом <tbody>, створеним браузером.

```
<tfoot>    <tr>       <td colspan = "4">This is the foot of the table</td>    </tr> </tfoot>
```

Елемент tr

Елемент HTML **<tr>** визначає ряд у таблиці. Клітинки рядка можна створити за допомогою комбінації елементів **<td> і <th>**.

Щоб забезпечити додатковий контроль над тим, як клітинки вміщуються в стовпці, <th> і <td> підтримують атрибут **colspan**, який дозволяє вказати, скільки стовпців має ширина клітинки, за замовчуванням 1. Подібним чином, ви можете використовувати атрибут **rowspan**, щоб вказати, що клітинки повинні охоплювати більше одного рядка таблиці.

Таблиця в HTML складається з рядків, а не стовпців. Кожна клітинка таблиці міститься в елементі <tr>. Зазвичай ці елементи є нащадками <thead>, <tbody> і <tfoot>, але також можуть бути прямими нащадками <table>, якщо елементи area не використовуються.

```
<thead>     <tr></tr>  </thead>
<tbody>     <tr></tr>  </tbody>
```

Елемент td

Практика: Таблиці

Елементи **<td>** містять окремі порції даних таблиці. **<th>** визначає вміст заголовка для рядка або стовпця. Елементи <th> і <td> поводяться як клітинки. Щоб у таблиці було чотири стовпці, кожен <tr> повинен мати не більше і не менше чотирьох елементів <td> або <th>. Залежно від вмісту даних це може означати наявність порожнього елемента <th> або <td>.

```
<table border = "1">
 <tr>
    <th>Row 1, Column 1</th>    <th>Row 1, Column 2</th>
 </tr>
  <tr>
    <td>Row 2, Column 1</td>    <td>Row 2, Column 2</td>
 </tr>
</table>
```

Заголовки, визначені в тезі <th>, вирівняні по центру та виділені жирним шрифтом.

Атрибут scope

Таблиці

Атрибут **scope** визначає клітинки, до яких відноситься елемент заголовка і може мати такі значення:

- **row**: заголовок стосується всіх клітинок рядка, до якого він належить.
- **col**: заголовок стосується всіх клітинок стовпця, до якого він належить.
- **rowgroup**: заголовок належить до групи рядків і стосується всіх її клітинок. Ці клітинки можна розмістити праворуч або ліворуч від заголовка, залежно від значення атрибута dir в елементі <table>.
- **colgroup**: заголовок належить до colgroup і стосується всіх її клітинок.

Якщо атрибут scope не вказано або його значення не є row, col, rowgroup або colgroup, браузери автоматично вибирають набір клітинок, до яких застосовується клітинка заголовка.

Атрибути cellpadding і cellspacing

Практика: Таблиці

Два атрибути **cellpadding** і **cellspacing** призначені для налаштування пробілу в клітинках таблиці. Атрибут cellspacing визначає відстань між клітинками таблиці, тоді як cellpadding представляє відстань між межами клітинки та вмістом у клітинці.

```
<table border = "1" cellpadding = "5" cellspacing = "5">
 <tr>
    <th>Name</th>
    <th>Salary</th>
 </tr>
 <tr>
    <td>Ramesh Raman</td>
    <td>5000</td>
 </tr>
 <tr>
    <td>Shabbir Hussein</td>
    <td>7000</td>
 </tr>
</table>
```

Атрибут colspan

Практика: Таблиці

Атрибут colspan використовують, якщо потрібно об’єднати два або більше стовпців в один.

```
<table border = "1">
 <tr>    <th>Column 1</th>    <th>Column 2</th>    <th>Column 3</th> </tr>
 <tr>    <td colspan = "3">Row 3 Cell 1</td> </tr>
<tr>
    <td rowspan = "2">Row 1 Cell 1</td>
    <td>Row 1 Cell 2</td>
    <td>Row 1 Cell 3</td>
 </tr>

</table>
```

Подібним чином використовується rowspan, якщо хочете об’єднати два чи більше рядків.

Властивість table-layout

Практика: Таблиці

Властивість table-layout встановлює алгоритм, який використовується для компонування комірок, рядків і стовпців <table>.

table-layout: auto;

Використовується алгоритм автоматичної верстки таблиці. Ширина таблиці та її комірок регулюється відповідно до вмісту. Більшість браузерів використовують цей алгоритм за замовчуванням.

table-layout: fixed;

Використовується алгоритм верстки фіксованої таблиці, ширину таблиці потрібно вказати явно за допомогою властивості **width**.

Алгоритм макета фіксованої таблиці є швидшим, ніж алгоритм автоматичного макета, оскільки горизонтальний макет таблиці залежить лише від ширини таблиці, ширини стовпців і меж або інтервалу між клітинками. Горизонтальний макет не залежить від вмісту комірок, оскільки він залежить лише від явно встановленої ширини.

Фіксоване компонування таблиц

Практика: Таблиці

В алгоритмі фіксованої компонування таблиці ширина кожного стовпця визначається наступним чином:

- Елемент стовпця з явною шириною встановлює ширину цього стовпця.
- Інакше клітинка в першому рядку з явною шириною визначає ширину цього стовпця.
- Інакше стовпець отримує ширину спільного горизонтального простору, що залишився.

За допомогою цього алгоритму всю таблицю можна відобразити після завантаження та аналізу першого рядка таблиці. Це може пришвидшити час візуалізації в порівнянні з автоматичним методом макета, але наступний вміст клітинки може не вміщатися в надану ширину стовпців.

Клітинки використовують властивість переповнення, щоб визначити, чи вирізати будь-який переповнений вміст, але лише якщо таблиця має відому ширину; інакше вони не переповнять клітинки.

Властивість display: table

Практика: Таблиці

Властивість **display: table** змушує елемент поводитися як таблиця. Ви можете створити копію таблиці без використання елемента table та відповідних елементів, таких як tr і td. Наприклад, ви можете створити таблицю з елементом <table>, а також або будь-яким контейнером на ваш вибір.

Елементи **table-row-group** поводяться як елементи <tbody>.
Елементи **table-header-group** поводяться як елементи HTML <thead>.
Елементи **table-footer-group** поводяться як елементи HTML <tfoot>.
Елементи **table-row** поводяться як елементи HTML <tr>.
Елементи **table-cell** поводяться як елементи HTML <td>.
Елементи **table-column-group** поводяться як елементи HTML <colgroup>.
Елементи **table-column** поводяться як елементи HTML <col>.

Створення макета таблиці

Практика: tables

Створення макета таблиці за допомогою Flexbox вимагає обробки контейнерів рядків як гнучких ліній, а комірок як гнучких елементів. Flexbox дуже корисне для створення адаптивних таблиць на основі карток або легкої реструктуризації порядку макета на менших екранах.

```
<div class="flex-table">
 <div class="table-row header">
   <div class="table-cell">Name</div>
   <div class="table-cell">Role</div>
   <div class="table-cell">Status</div>
 </div>
<div class="table-row">
   <div class="table-cell">Alice Vance</div>
   <div class="table-cell">Lead Designer</div>
   <div class="table-cell">Active</div>
 </div>
 <div class="table-row">
   <div class="table-cell">Bob Smith</div>
   <div class="table-cell">Developer</div>
   <div class="table-cell">Away</div>
 </div>
</div>
```

Контейнер-обгортка

Практика: tables

```
/* The wrapper container */
.flex-table {
 display: flex;
 flex-direction: column;
 border: 1px solid #ddd;
 border-radius: 4px;
 overflow: hidden;
}
```

Кожен рядок є горизонтальним гнучким контейнером

Практика: tables

```
/* Each row is a horizontal flex container */
.table-row {
 display: flex;
 flex-direction: row;
 width: 100%;
 border-bottom: 1px solid #ddd;
}
```

Видалити рамку з останнього рядка

Практика: tables

```
/* Remove border from the last row */
.table-row:last-child {
 border-bottom: none;
}
```

Стиль, специфічний для заголовка

Практика: tables

```
/* Header specific styling */
.table-row.header {
 background-color: #f5f5f5;
 font-weight: bold;
}
```

Клітини розтягуються та стискаються однаково

Практика: tables

```
/* The cells stretch and shrink equally */
.table-cell {
 flex: 1;
 padding: 12px;
 text-align: left;
 /* Visual separator for columns (optional) */
 border-right: 1px solid #ddd;
}
```

Видалити праву межу з останнього стовпця

Практика: tables

```
/* Remove right border from the last column */
.table-cell:last-child {
 border-right: none;
}
```

Зауваження щодо таблиць Flexbox

Практика: tables

На відміну від нативних таблиць HTML, макети Flexbox не враховують багатовимірні обмеження. Під час проєктування пам’ятайте про такі характеристики:\

Суворе вирівнювання стовпців:

Щоб ширина стовпців була однаковою для кількох рядків, призначте відповідну ширину за допомогою flex: 1 (однакові розміри) або явно встановіть відсотки (наприклад, flex: 0 0 25%) для певних дочірніх індексів.

Імітація Colspan: Ви можете імітувати макет colspan="2", помноживши значення flex-grow (наприклад, .wide-cell { flex: 2; }) на будь-який елемент комірки.

Адаптивне згортання: На екранах мобільних пристроїв ви можете повністю порушити парадигму таблиці, написавши швидкий медіа-запит, щоб змінити напрямок рядка (.table-row { flex-direction: column; }), перетворивши комірки таблиці на незалежні стекові блоки.

ВПРАВИ

Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Vs Code : https://code.visualstudio.com/

Webstorm: https://www.jetbrains.com/webstorm/

Web technology For developers https://developer.mozilla.org/en-US/docs/Web

CSS: https://developer.mozilla.org/en-US/docs/Web/CSS

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

Консорціум Всесвітнього павутиння https://www.w3.org/

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua